

Reinforcement Learning for Bomberman

Luis David Maco Cangalaya Diana Krasnokutskaya Daniela Ardila Porras

Machine Learning Essentials – Summer Semester 2026

1 Introduction

Bomberman is a simple game with a difficult decision problem. In the version used for this project, four agents move on a 17×17 grid of walls, crates and free tiles for at most 400 steps. In each step an agent picks one of six actions: four moves, waiting, or dropping a bomb. A bomb explodes after four steps and destroys everything within three tiles in a straight line until a wall stops the blast. Crates can hide coins. A coin is worth one point, and blowing up an opponent is worth five. The agent has 0.5 seconds of CPU time per step to decide what to do. Our goal was to train an agent with reinforcement learning (RL) that plays this game as well as possible, and in particular one that beats the provided `rule_based_agent` on the `classic` board.

We worked through four tasks of increasing difficulty:

- (1) collecting coins on an empty board
- (2) destroying crates and finding hidden coins without killing oneself
- (3) hunting the provided `peaceful_agent` and `coin_collector_agent`
- (4) holding one's own against the full-strength `rule_based_agent`.

Tasks 1–3 serve as stepping stones and debugging stages. Task 4 is the criterion the tournament actually rewards.

The task is small and cheap to simulate, but still includes many of the challenges that make reinforcement learning difficult in practice.

- **Delayed and sparse credit.** Rewards are rare. An agent that walks into its own blast dies four steps after the decision that actually killed it, and the step that traps an opponent happens well before the opponent dies. Credit has to travel back through several steps.
- **Safety-critical actions.** One wrong bomb ends the round. For every agent we trained, a large share of all losses came from suicides rather than from opponents.
- **A non-stationary, multi-agent environment.** From task 3 on, other agents drop bombs and compete for the same coins, so a safe state can become deadly because of someone else's decision.
- **A large state space.** Tabular Q-learning over raw boards is impossible. We either design a compact feature representation by hand or let a neural network learn one.
- **Limited compute and time.** Three weeks, one CPU thread per agent in the tournament.

We developed two structurally different agents:

1. **q_agent**, a linear Q-learning agent over 34 hand-engineered features (walkability, danger countdowns, BFS directions to coins, crates, safe tiles and opponents, and a few bomb-specific indicators). It uses an action mask that removes illegal and provably suicidal actions, and a hand-designed set of auxiliary rewards.
2. **dqn_agent**, a Double Deep Q-Network with a target network and experience replay. It reads a six-channel image of the board (walls, crates, self, others, coins, danger) through a convolutional encoder and learns its own features. It deliberately contains no path-finding or danger heuristics in its state encoding.

The two agents allow us to compare two choices under the fixed time budget: putting domain knowledge into hand-crafted features or letting a network learn the representation.

2 Background

3 Project Planning

Author: Luis Maco

We are a team of three (Diana Krasnokutskaya, Daniela Ardila Porras, Luis Maco) working on the final project for Machine Learning Essentials, Summer Semester 2026. The project ran from registration in the week of 31.08 to the report deadline on 28.09, with an optional crash test on 17.09 and code due on 21.09.

The plan

The specification prioritises systematic testing over tournament placement. We therefore recorded a before/after number for every design change, built an evaluation harness before any learning agent existed, and kept a dated experiment log (**experiments/LOG.md**) from day one so the results section could be assembled from files rather than from memory.

The specification required two models and emphasised shared work. We initially planned a component-based split covering the environment interface, learning algorithm, evaluation, and experiments, with shared feature and evaluation infrastructure. In practice, each model was carried end to end by a different team member, while evaluation, event definitions, and reward-shaping principles were shared and cross-checked. This model-based split proved easier to maintain than a strict component-based split.

We planned two structurally different models: a linear Q-learning agent over hand-engineered features as the primary tournament candidate, and a Deep Q-Network as a comparison. The DQN was allowed to fail; negative or mixed results (Section 6.1) were part of the planned comparison rather than a reason to hide the approach.

We planned a four-task curriculum with numeric gates for coin collection, safe crate bombing, hunting weaker agents, and beating **rule_based_agent**. Task 2 was the critical path because escaping bombs is crucial and most tournament losses are self-inflicted, so we placed it in week 1 to leave time for iteration.

How it actually went

Both agents reached the Task 1 gate on 06.09: the DQN collected 46.06 of 50 coins (95% CI [45.5, 46.7], zero invalid actions), and the feature-based agent matched the **coin_collector_agent** baseline. Task 2 took three reward-function fixes and roughly four days before the DQN's suicide rate on **classic** passed the 2% gate.

On 06.09. we agreed not to plot training-time measurements because the online and frozen (`train=False`) versions of the same weights can disagree substantially. This discrepancy was observed on the DQN side, but the evaluation rule applied to both agents because the tournament always runs a frozen policy. On 15.09 we built a shared comparison tool (`tools/compare_team_agents.py`) that evaluates both agents frozen, against the same seeded opponents under one protocol. Confirmed at $N=1000$ rounds on 16.09, that comparison selected the DQN (`score_hunt r1250`) as the team’s tournament candidate over the committed feature-based checkpoint at that time. The DQN arm froze on 16.09, four days ahead of schedule, after a controlled sweep (Section 6.1) showed that seed variation exceeded every tested hyperparameter effect. We spent the remaining days confirming the result and writing it up.

Hardware

Neither model needed a GPU. The DQN has 353k parameters, only about 6k of them convolutional, and profiling (Section 5.1) showed that gradient steps dominate the measured cost. Training and sweeps ran on one Apple M3 Max laptop with 36 GB RAM, using one CPU thread per process. Both agents also respect the tournament’s CPU-only 0.5 s-per-step budget.

4 Methods

4.1 The DQN agent

Author: Luis Maco

State representation

`dqn_agent` reads six board channels: walls, crates, self, opponents, coins, and danger countdowns. The channels remain fixed across tasks, including the channels that are empty on Task 1. The board is represented either globally at 17×17 or as a padded 13×13 window centred on the agent (`ego:13`); this choice is stored with the checkpoint because the tournament provides no environment variables.

Network and learning rule

The agent is a Double DQN [2] with a target network synchronised every 1000 gradient steps and a uniform replay buffer of 50,000 transitions. Double DQN separates action selection from evaluation in the bootstrap target, reducing the overestimation bias of vanilla DQN [2]. A small convolutional front end feeds a fully connected head with one output per action.

Actions and masking

All six actions are available at the architecture level. A legal-action mask removes moves into blocked or immediately lethal tiles and is also applied to the Double-DQN target. An optional suicide mask removes BOMB when breadth-first search finds no safe escape within the four-step fuse; from Task 3 onward it also considers tiles an opponent can reach first. The suicide mask is enabled only at play time, while the legal-action mask is always active.

Episode boundaries

A round that reaches the 400-step limit is treated as a truncation, following Pardo et al. [4]. It is not assigned a terminal target with value zero, which previously combined with potential shaping to reward being far from a coin at timeout.

Reward shaping

The base reward is sparse, so `dqn_agent` adds potential-based shaping and custom events. Following Ng, Harada and Russell [3], it uses $F(s, s') = \gamma_{\Phi} \Phi(s') - \Phi(s)$, with $\Phi(s)$ the negative breadth-first-search distance to the nearest reachable coin (or, once no coin is visible, to the nearest tile from which a bomb would break a crate). Under the standard potential-based shaping conditions, this form leaves the optimal policy unchanged when γ_{Φ} equals the agent’s discount factor γ . We deliberately break that equality (Section 5.1) and cap the potential, so the exact invariance guarantee no longer applies; these changes removed a reward exploit observed in practice.

The event rewards distinguish useful, useless, and suicidal bombs, while danger events shape escape. They use information already available in the game state and event list, so they fit within the 0.5s step budget.

The training profiles are `curriculum` for Tasks 1–2, `score` for scoreboard events plus safety shaping, and `score_hunt`, which adds a bonus for an escapable bomb threatening an opponent. We introduced them after observing a mismatch between training reward and the tournament scoreboard.

Symmetry augmentation

The `classic` board is invariant under eight square symmetries. Augmentation is applied when sampling from replay, per sample rather than per batch, so the original transition is stored once and transformed orientations are sampled independently. It affects training only. Action permutations were checked by comparing transformed states with recomputed features for all eight group elements.

Evaluation methodology

All DQN results use frozen policies (`train=False`). Comparisons use fixed seeds, $N \geq 200$ rounds, and $N = 1000$ near gate thresholds; a gate passes only when its whole 95% confidence interval clears the threshold. Because `round_statistics` aggregates all agents, per-agent statistics are used when opponents are present.

5 Training

5.1 The DQN agent

Author: Luis Maco

Curriculum

Training followed the four-task curriculum, resuming weights between stages. On Task 1 (`coin-heaven`), BOMB and WAIT were removed from exploration because an $\varepsilon=1$ policy could otherwise die before collecting experience. Task 1 does not require bombs, so the mask was lifted for Task 2. We also used a temporary `crate-light` scenario with crate density 0.35 instead of the tournament’s 0.75; it exists only in `settings.py`.

Opponent choice as a training variable

From Task 3 onward, opponent choice had a substantial effect. Two 4000-round runs resumed from the same checkpoint and differed only in the opponent: the agent scored 81 kills, flat over time, against `coin_collector_agent`, but 258 against `rule_based_agent`, rising from 0.010 to 0.147 kills per round. The former ignores the agent; the latter approaches and bombs it. We

therefore used three copies of `rule_based_agent` as the Task 4 training partners rather than the easier opponents used earlier in the curriculum.

Training efficiency

Profiling one training round showed gradient steps at 92% of the cost, compared with 5% for acting and 2% for feature extraction. We therefore stored replay data as `uint8` instead of `float32`, changed updates from 32 samples every 4 steps to 128 every 16, and ran ablations as separate single-threaded processes. Six configurations covering roughly 7.2 million environment steps then completed in 51 minutes instead of an estimated five hours sequentially.

Reward shaping problems found and fixed during training

Two reward problems surfaced during training. The potential-based shaping term, computed with the same discount as the agent’s $\gamma = 0.95$, leaves a residual of $\Phi(s)(\gamma - 1)$ on every step that makes no progress; at a coin ten steps away this residual paid roughly $+0.065$ every two steps for oscillating in place, for as long as the agent chose to oscillate, and once the next coin was twelve or more steps away, collecting the current one was net negative under the shaping term. The fix decouples the shaping discount from γ (set to 1.0, which removes the per-step drift so the step penalty makes hovering strictly negative) and caps the distance term, bounding the size of the jump when a coin is collected. Separately, once crates entered the curriculum, the potential function’s target (the nearest reachable coin) was undefined on every Task 2 board, because every coin starts hidden inside a crate. The Task 1 shaping signal was therefore absent on Task 2 until the potential was retargeted to the nearest good bombing spot when no coin is visible. Both fixes were checked against the arithmetic before another training run was started: a five-step expected-value comparison of “bomb two crates” against “do nothing” moved from -0.44 to $+2.67$ once the four Task 2 reward fixes (including suppressing the “entered danger” penalty on the step a bomb is deliberately dropped) were applied together.

Checkpointing and instability

Checkpoints were evaluated frozen, and the best rather than the last checkpoint was kept. Most from-scratch egocentric-view runs peaked while ε was still around 0.84, then decayed as invalid actions rose from 0 to 40–80 per round. This is consistent with online-DQN instability. The global-view run improved monotonically, leaving the egocentric crop as a possible contributor and an open problem (Section 6.1).

Reward-profile progression toward the tournament objective

The Task 2 curriculum profile paid for crate destruction that had no direct scoreboard equivalent: `CRATE_DESTROYED` contributed roughly $+24$ training reward per round. The `score` profile was safer but did not meet the tournament objective. We therefore added `score_hunt`, a bonus for an escapable bomb threatening an opponent. Hand-built boards verified that it did not fire for suicidal bombs or crate-only demolition, and the profile improved the tournament margin (Section 6.1).

Fine-tuning

When a policy overfit to one training distribution and lost general ability (Section 6.1), a short, low-learning-rate fine-tune from the same checkpoint was sufficient: 1500 solo rounds at one fifth of the normal learning rate, 7.6 minutes of wall clock, recovered solo coin collection from 4.95 to 7.87 coins while keeping the tournament gate the model had already passed.

6 Experiments and Results

6.1 The DQN agent

Author: Luis Maco

All numbers below are measured frozen (`train=False`), with $N \geq 200$ rounds and a fixed seed list unless noted, and a gate counts as passed only when the whole 95% confidence interval clears its threshold.

Baselines

On `coin-heaven`, an untrained network acting under an ϵ -greedy random policy averages about 18 coins with roughly 145 invalid actions per 400-step round; `rule_based_agent` averages 50.0 coins in 125.2 steps with zero invalid actions. On solo `classic`, `rule_based_agent` averages 8.60 coins with 0% suicide; in the four-player tournament shape (three copies of `rule_based_agent`) it averages a 3.30 mean score with 6.9 invalid actions per round of its own.

Gate results

Task	Gate	Result	Value	N
1, <code>coin-heaven</code>	$\geq 45/50$ coins, 0 invalid	PASS	46.06 coins, CI [45.5, 46.7]	200
2, <code>classic</code> survival	suicide $< 2\%$	PASS	0.00%, 95% upper 0.38%	1000
2, <code>classic</code> coins	$\geq 8/9$ coins	FAIL	2.37 ± 0.12	1000
3, vs <code>peaceful_agent</code>	kill in $\geq 80\%$ of rounds	FAIL	3.0% [95% 2.1, 4.3]	1000
3, vs <code>coin_collector_agent</code>	positive margin	PASS	$+0.89 \pm 0.35$	400
4, vs $3 \times \text{rule_based_agent}$	mean score strictly above	PASS	4.961 ± 0.202 , margin $+2.250 \pm 0.238$	1000

Table 1: Final candidate (`score_hunt r1250`) against every curriculum gate. Task 4 is the specification’s requirement; Tasks 1–3 are our own curriculum checkpoints.

Task 1 and Task 4 pass their gates. The selected checkpoint is weaker on the Task 2 solo test and against the passive opponent in Task 3. An earlier checkpoint (`task4-C-solo-refresh-r1000`, called *C* below) reached 7.70 ± 0.11 solo coins and 76.7% kill-in-round against `peaceful_agent`, closer to those gates, but its tournament margin against the weakest of three `rule_based_agents` was only $+0.54 \pm 0.28$, against $+2.18 \pm 0.27$ for the selected checkpoint. Both pass Task 4, while checkpoint C performs better on the solo and passive-opponent tests.

Measurement and implementation bugs

Four bugs were identified through measurement.

1. **Exploration suicide.** With BOMB in the random exploration set, the agent killed itself within a few steps on `coin-heaven`, ending training before learning began. We masked BOMB for Task 1.
2. **Online policies are not frozen policies.** The same saved weights scored 45 coins online while the network was updating and 6.2 frozen, a factor of seven. All subsequent evaluations therefore use frozen policies.
3. **Reward shaping paid for idleness.** Covered in Section 5.1: the discounted potential term rewarded oscillating near a coin and made collecting a distant coin net negative in the worst case.

4. **Cached next_state equalled state.** A transition cache keyed on (`round`, `step`) collided, because the environment increments its step counter once, before an agent acts, so the state passed to `act()` and its successor carried the same key. Instrumentation showed that 800 of 800 sampled transitions had `next_state == state` before the fix, and 35.2% after, consistent with moves that leave the agent in the same position. With $s' = s$, the Bellman target becomes $r + \gamma \max_a Q(s, a)$ for the same state, preventing value from propagating correctly. Wall avoidance was learned almost perfectly, while multi-step coin collection was not.

Before the fourth bug was fixed, a six-configuration ablation varying shaping, view, learning rate, and batch settings placed all configurations between 2.95 and 5.20 frozen coins with overlapping intervals. This lack of separation led to the transition-cache bug.

Specialization under training against `rule_based_agent`

A checkpoint trained purely against three `rule_based_agents` (`task4-classic-r3250`) passed Task 4 with a $+0.78 \pm 0.25$ margin at $N=1000$. Against the other boards, it lost ground relative to its Task 2 parent: solo `classic` coins fell from 7.51 to 4.34, and kill-in-round against `peaceful_agent` fell from 69.2% to 37.7%, while its self-kill rate did not get worse (0.60% $\rightarrow$ 0.50% solo; 42.5% $\rightarrow$ 33.2% in the crowded game). The bombing rate also fell, from 47.6 bombs per solo round for the parent to 26.5 for this checkpoint. The four thousand rounds against `rule_based_agents` are consistent with caution that reduces crate destruction and access to hidden coins in solo play. This suggests opponent-distribution overfitting on the same seven boards. The final `score_hunt` checkpoint shows the same pattern. Its bombing rate depends on the opponent: it is 44.2 bombs per round against three `rule_based_agents`, 32.5 against `coin_collector_agent`, and only 12.5 both solo and against `peaceful_agent`. This is consistent with its stronger performance on the required gate and weaker performance on the solo-coin and peaceful-hunting curriculum tests that earlier checkpoints had passed. Task 4 is the specification’s requirement; Tasks 1–3 are the project’s curriculum checkpoints.

Hyperparameter sweep and seed sensitivity

Six from-scratch configurations were trained for 5000 rounds on `classic` against three `rule_based_agents` and ranked over the last five checkpoints (300 pooled evaluation rounds each). Three changed one hyperparameter relative to control; a fourth used the global view, and the control was repeated with another seed.

Configuration	Pooled margin	Suicide	Kill/round
control (seed A)	−1.401	31.0%	12.0%
augmentation	−2.008	46.7%	12.0%
target sync every 250	−2.190	46.7%	12.0%
discount 0.99	−2.354	53.0%	9.7%
control (seed B)	−2.431	54.0%	10.7%
global view	−4.117	6.0%	1.0%

Table 2: Task-4 hyperparameter sweep, ranked on pooled margin over the last five checkpoints of a 5000-round from-scratch run.

The two controls differ by 1.030 margin points. The augmentation, target-sync, and discount configurations all fall between the two observed control results, at margins from −2.354 to −2.008. The global-view configuration falls outside this range at −4.117, 1.686 points below the worse control. This differs from the Task 1 result, where the global view was the only configuration that kept improving instead of decaying. Its low 6.0% suicide rate does not by itself indicate learned bomb safety: the same configuration scored 0.15 and killed an opponent in only 1% of rounds.

No from-scratch run closed the roughly two-point gap to the incumbent within budget. The runs had not clearly flattened by round 5000; the control’s margin moved $-3.17 \rightarrow -3.02 \rightarrow -2.22 \rightarrow -1.12$ across the run. The results therefore do not establish that the tested factors are unimportant with longer training. We stopped after the sweep because no tested configuration had matched the already-confirmed incumbent within the available budget.

Selection

`score_hunt r1250` was chosen as the final DQN tournament candidate on Task 4, the specification’s required gate. It was shortlisted from a 40-round-per-checkpoint curve, confirmed at $N=200$ on an unseen seed, and re-confirmed at $N=1000$ on a second unseen seed. Its three individual paired margins against the three `rule_based_agents` are $+2.290 \pm 0.265$, $+2.281 \pm 0.271$, and $+2.178 \pm 0.267$; every 95% lower bound is above $+1.9$. Checkpoint *C* also passes Task 4 with a smaller positive margin and performs better on the solo and passive-opponent tests reported above, but has a higher crowded-room suicide rate (32.5% against 10.9%).

Controlled comparison with the feature-based agent

Using the shared frozen comparison protocol (`tools/compare_team_agents.py`), both agents were evaluated separately against the same seeded three-`rule_based_agent` opponents at $N=200$ on seed 157. The DQN scored 4.615 ± 0.385 against the feature-based agent’s 3.065 ± 0.320 , with a mean margin over the three opponents of $+1.858 \pm 0.465$ against $+0.233 \pm 0.403$, and was strictly first in 43.0% of rounds against 23.5%. Under this protocol, the DQN passed the required Task 4 gate and the feature-based checkpoint tested at that time did not.

The DQN’s Task 4 result was later confirmed at $N=1000$ on a separate seed, as reported above. The feature-based agent was not re-evaluated at that sample size within the project timeline, so the tournament-entry decision used the $N=200$ controlled comparison. The tested feature-based checkpoint was the checkpoint available for comparison at the time; it was not independently verified as the owner’s final intended submission.

7 Conclusion

Work distribution

WP	Topic	Lead
WP1	Data pipeline and preprocessing	D. Krasnokutskaya
WP2a	Dense autoencoder	D. Krasnokutskaya
WP2b	Convolutional autoencoder	J. Mutluer
WP3	Optimizers, training loop, metrics	J. Mutluer
WP4	Latent-space and robustness analysis	D. Krasnokutskaya
WP5	Poster and report	both

References

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, et al. *Human-level control through deep reinforcement learning*. *Nature*, 518(7540):529–533, 2015.
- [2] H. van Hasselt, A. Guez, and D. Silver. *Deep Reinforcement Learning with Double Q-learning*. *Proc. AAAI Conference on Artificial Intelligence*, 2016.

- [3] A. Y. Ng, D. Harada, and S. Russell. *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*. Proc. 16th International Conference on Machine Learning (ICML), 1999.
- [4] F. Pardo, A. Tavakoli, V. Levдик, and P. Kormushev. *Time Limits in Reinforcement Learning*. Proc. 35th International Conference on Machine Learning (ICML), 2018.